CENTRAL TENDENCY, VARIATION, AND OUTLIERS

Learning objectives

- Using an appropriate software tool, calculate:
  - Mean, Mode, Median, Mid-range
  - Population and Sample Standard Deviation and Variance
  - Inter-Quartile Range
- Discuss when the above measures are appropriate
- Apply methods for automating identification of outliers
- Discuss appropriate handling of outliers

Expected prior knowledge

- Minimum of GCSE Maths or equivalent
- Experience with Python or R programming for Data Handling

Anaconda and Jupyter

Anaconda is a collection of Python packages, programs, and other software for data analysis and data science.

A Jupyter notebook is a documentation or commentary-first style of programming, where the key elements are explanations and comments. The code is less important, and often only small amounts.

JUPYTER BASICS

How do I start Anaconda?

Start Menu > Anaconda Navigator

How do I start Jupyter notebook?

In Anaconda Navigator, press LAUNCH underneath the Jupyter notebook icon.

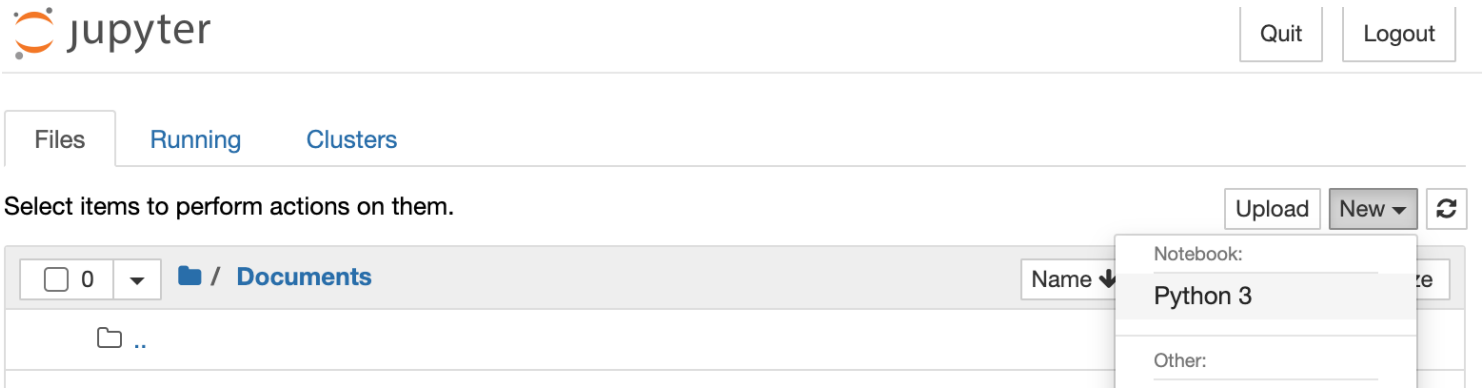
OPENING JUPYTER

How do I use the file browser?

Jupyter notebook starts with a file browser that displays files on your system, eg., browse to your documents folder in Jupyter.

How do I create notebooks?

In your documents folder, go to the **New** button, and press Python3.

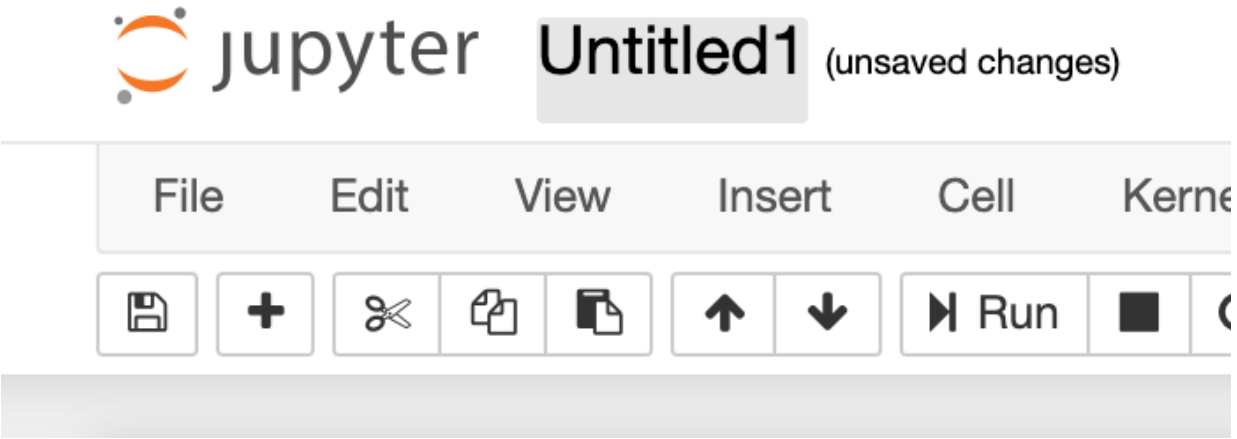


Jupyter will create a new notebook called ‘Untitled’.

RENAMING NOTEBOOKS

How do I rename a notebook?

Click on the title and a renaming box will appear. Choose a relevant title, e.g., ‘MyNotes’.

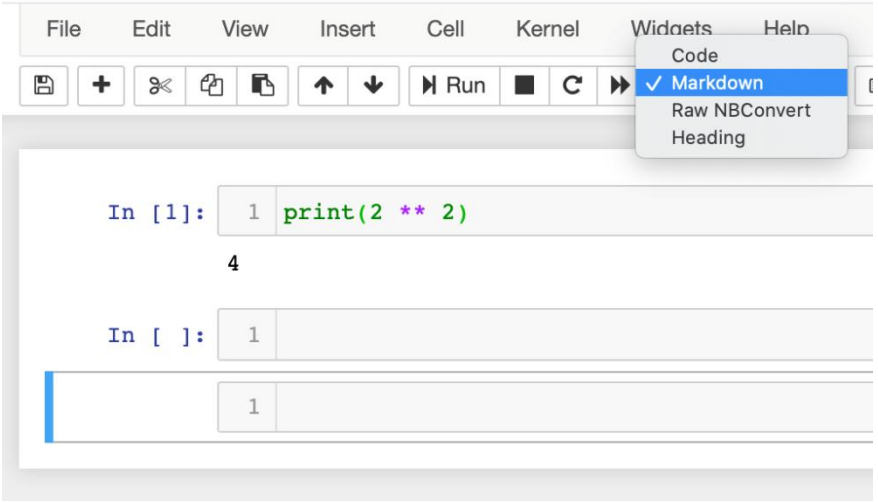


WORKING WITH CELLS

Code cells

Python code can be typed in a code cell – to run the code, either press the 'Run' button or use CTRL + ENTER.

Jupyter will display the last line of un-assigned data without the need for the print() command.



Text cells

Add a cell, change type to markdown. When these are 'Run' they will be marked up.

DATA BASICS: TABLES

Variables

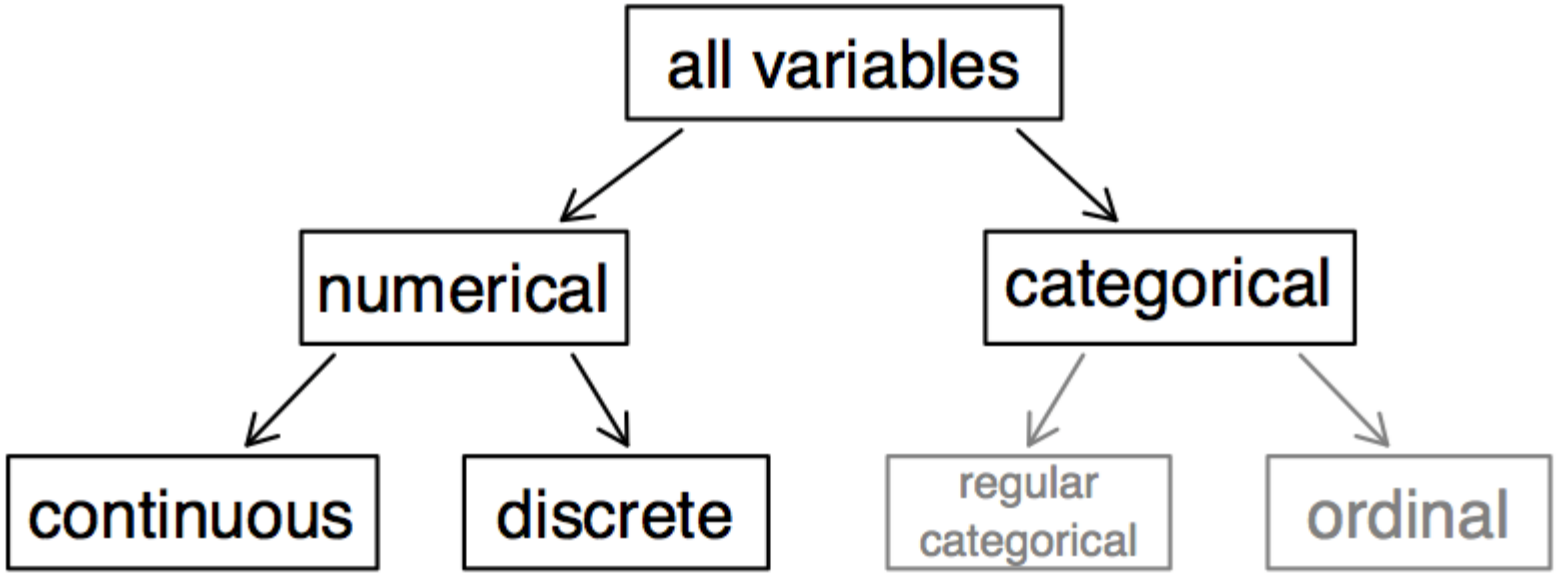
Each variable is a column with a unique header

Column headers may also be referred to as attributes or features.

Data within a column is the same data type.

Each row is called an **observation** or record.

DATA BASICS: TYPES OF VARIABLES



MEASURES OF CENTRAL TENDENCY (AVERAGES!)

e.g.,
2, 5, 4, 5, 3

Mean

$\bar{x}$ = (2+5+4+5+3)/5 = 3.8

Mode

Frequency	
2:	1
3:	1
4:	1
5:	2

Median

2, 3, 4, 5, 5

Mid-range

(2+5)/2 = 3.5

MEASURES OF CENTRAL TENDENCY: PYTHON

```python
import pandas as pd

data = pd.Series([2, 5, 4, 5, 3])

mean = data.mean()
mode = data.mode().iloc[0]
median = data.median()
mid_range = (data.min() + data.max()) / 2

print("Mean:", mean)
print("Mode:", mode)
print("Median:", median)
print("Mid-Range:", mid_range)
```

```
Mean: 3.8
Mode: 5
Median: 4.0
Mid-Range: 3.5
```

GROUPED DATA

e.g.

Age	Frequency
10 <= x < 20	5
20 <= x < 30	8
30 <= x < 40	12
40 <= x < 50	7
50 <= x < 60	3

If there are no gaps between the groups, we refer to them as **classes**.

The data can now be treated as **continuous** (even if ungrouped data was originally discrete).

The fine detail of the original data has been lost – to calculate summary statistics we need to **make an assumption** that means our results will be **estimates**.

When we need to, we **assume** the data is **evenly spread within each class** – this could be referred to as **uniformly distributed**.

GROUPED DATA: MODAL CLASS AND MID-RANGE

Modal Class:

e.g.,

Age	Frequency
10 <= x < 20	5
20 <= x < 30	8
30 <= x < 40	12
40 <= x < 50	7
50 <= x < 60	3

Mid-range estimate: (10+60)/2 = 35

GROUPED DATA: MEDIAN

e.g.

Age	Frequency
10 <= x < 20	5
20 <= x < 30	8
30 <= x < 40	12
40 <= x < 50	7
50 <= x < 60	3

Number of pieces of data: 5+8+12+7+3 = 35

Halfway: 17.5

The first 5 pieces of data in the first class start at Age 10 and are assumed to be evenly spaced until the end of the class at Age 20. By Age 30 we would reach the 13th piece of data.

We need to get to 17.5 so need to go another 4.5 pieces of data through the third age group.

Proportion of the way through the group: 4.5/12 = 0.375

The class is 10 wide so to get to an **estimate of the median** from the start of that class (Age 30) we reach **33.75**

GROUPED DATA: MEAN

e.g.

Age Mid-Point	Frequency
15	5
25	8
35	12
45	7
55	3

An **estimate of the mean** is found by using the mid-point of each class (this is what the values would average to if we assume they are uniformly distributed within the class).

The total of the ages in the first group is 15 x 5 = 75.

We repeat the calculation for all groups and then find a grand total: 15x5 + 25x8 + 35x12 + 45x7 + 55x3 = 1175

$\bar{x}_{estimated}$ = 1175/35 = 33.6 (3 s.f.)

GROUPED DATA MEAN: PYTHON

```python
grouped_data = [(10, 5), (20, 8), (30, 12), (40, 7), (50, 3)]

# Create a DataFrame from grouped data with a column for mid-points
df = pd.DataFrame(grouped_data, columns=['Value', 'Frequency'])
df['Midpoint'] = [15, 25, 35, 45, 55]

# Create a Series with repeated midpoints based on frequencies
data = pd.Series(df['Midpoint'].repeat(df['Frequency']))

mean = data.mean()

print("Class Midpoints:")
print(df)
print("Mean:", mean)
```

```
Class Midpoints:
   Value  Frequency  Midpoint
0     10          5        15
1     20          8        25
2     30         12        35
3     40          7        45
4     50          3        55
Mean: 33.57142857142857
```

MEASURES OF VARIATION: RANGE AND IQR

e.g.
2, 3, 4, 6, 7, 7, 8, 19

Range:

Width of spread for the whole data set (largest – smallest).

19 – 2 = 17

Inter-Quartile Range

Width of spread for the middle half of the data set.

Like finding the median, we find the first (or lower) quartile and the third (or upper) quartile before finding the difference.

IQR = Q3 – Q1
= 7.5 – 3.5
= 4

MEASURES OF VARIATION: STANDARD DEVIATION AND VARIANCE (POPULATION)

e.g.
2, 3, 4, 6, 7, 7, 8, 19

$x_i - \bar{x}$ is the distance from each piece of data to the mean.

Some distances are positive, and some are negative. Before calculating the average distance of the data from the mean we need to deal with the signs.

For standard deviation and variance this is done by squaring each of the distances first.

Variance, s^2, is the average of these squared distances from the mean. Standard Deviation, s, is the result of completing the calculation by square rooting.

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n} = 24.5$$

$$s = 4.95 \text{ (3 s.f.)}$$

MEASURES OF VARIATION: PYTHON

```python
data = [2, 3, 4, 6, 7, 7, 8, 19]
df = pd.Series(data)

Range = df.max() - df.min()

Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1

Variance = df.var(ddof=0)  # ddof=0 for population variance
StDev = df.std(ddof=0)    # ddof=0 for population standard deviation

print("Data:", df.tolist())
print("Range:", Range)
print(f"Interquartile Range (IQR): {Q3} - {Q1} =", IQR)
print("Population Variance:", Variance)
print("Population Standard Deviation:", StDev)
```

```
Data: [2, 3, 4, 6, 7, 7, 8, 19]
Range: 17
Interquartile Range (IQR): 7.25 - 3.75 = 3.5
Population Variance: 24.5
Population Standard Deviation: 4.949747468305833
```

QUARTILE DIFFERENCES: PYTHON

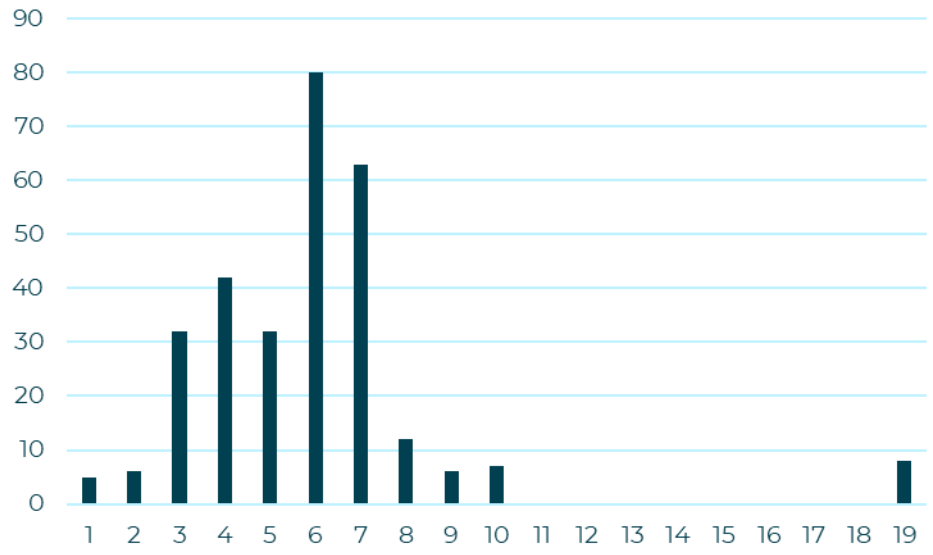
Pandas uses a method called linear interpolation for calculating quartiles by default, which can result in slight differences compared to manual calculations.

If you want to calculate quartiles in the same way we saw earlier, we will need to use numpy.percentile with the method set to ‘midpoint’.

```
import numpy as np

data = [2, 3, 4, 6, 7, 7, 8, 19]

q1 = np.percentile(data, 25, method='midpoint')
q3 = np.percentile(data, 75, method='midpoint')

print("Q1:", q1)
print("Q3:", q3)
```

```
Q1: 3.5
Q3: 7.5
```

OUTLIERS: VISUALLY



The importance of **investigating the outliers**

Keep it, Correct it, or Remove it?

1) The values are ages of people at a party
2) The values are scores from a test out of 10

OUTLIERS: CALCULATED

e.g.
2, 3, 4, 6, 7, 7, 8, 19

These two methods are designed to attempt to create boundaries which capture around 95% of the data typically – leaving just 5% plus outliers to be investigated if your data was normally distributed.

$$\bar{x} \pm 2s$$

= -3.58, 17.6 => outlier: 19

$$Q_1 - 1.5IQR$$
$$Q_3 + 1.5IQR$$

= -2.5, 13.5 => outlier: 19

Either could be used to automatically detect or highlight outliers.

OUTLIERS: PYTHON

```python
import numpy as np

data = [2, 3, 4, 6, 7, 7, 8, 19]

# Method 1: Mean plus or minus 2 standard deviations
mean = np.mean(data)
stddev = np.std(data)

lower_bound_method1 = mean - 2 * stddev
upper_bound_method1 = mean + 2 * stddev

outliers_method1 = [x for x in data if x < lower_bound_method1 or x > upper_bound_method1]

# Method 2: Q3 + 1.5 * IQR and Q1 - 1.5 * IQR
q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1

lower_bound_method2 = q1 - 1.5 * iqr
upper_bound_method2 = q3 + 1.5 * iqr

outliers_method2 = [x for x in data if x < lower_bound_method2 or x > upper_bound_method2]

print("Data:", data)
print("Method 1 Outliers:", outliers_method1)
print("Method 2 Outliers:", outliers_method2)
```

```
Data: [2, 3, 4, 6, 7, 7, 8, 19]
Method 1 Outliers: [19]
Method 2 Outliers: [19]
```

EXERCISE: PYTHON

- Import pandas as pd
- Create a list of numbers: data = [...] and use pd.Series(data) to save it as df.
- Use the method .mean(), .var(), and .std() to calculate the mean, variance, and standard deviation of your data.
- Calculate the outlier boundaries using mean – 2 * sd and mean + 2 * sd
- Check if there are any outliers in your data (values that are smaller than the lower bound or larger than the upper bound).

Example solutions: Python

```python
import pandas as pd

data = [2, 3, 4, 6, 7, 7, 8, 19]
df = pd.Series(data)

mean = df.mean()
print('Mean:', mean)

Variance = df.var(ddof=0)  # ddof=0 for population variance
StDev = df.std(ddof=0)    # ddof=0 for population standard deviation
print("Population Variance:", Variance)
print("Population Standard Deviation:", StDev)

lower_bound_method1 = mean - 2 * stddev
upper_bound_method1 = mean + 2 * stddev
outliers_method1 = [x for x in data if x < lower_bound_method1 or x > upper_bound_method1]
print('Outliers:',outliers_method1)
```

```
Mean: 7.0
Population Variance: 24.5
Population Standard Deviation: 4.949747468305833
Outliers: [19]
```

EXERCISE AND DISCUSSION:

WHICH MEASURES TO USE?

Which measure would best answer the question ‘What’s a typical value’ in each of these scenarios?

1. Test scores in a class
2. Eye colour in a survey of a population
3. Income of employees in a company
4. Age of participants in a marathon
5. Transportation used by people in a city
6. Height of individuals in a study
7. Products purchased in a store

EXERCISE

1. Use pd.read_csv() to save the investment_performance csv file as a pandas dataframe named df.
2. View the data. The scores are results from two investment strategies – the control group (using the current method) and the experimental group (using a new strategy).
3. Using df.describe() or otherwise, compare one measure of the 'typical' value for the two groups and one measure of how consistent or spread out the data is within each group.

Example solutions: Python

```python
import pandas as pd
import numpy as np

# Load data from the CSV file using pandas
csv_filename = 'investment_performance.csv'
df = pd.read_csv(csv_filename)

# Calculate statistics for 'Control Scores'
control_mean = df['Control Scores'].mean()
control_mode = df['Control Scores'].mode().iloc[0]
control_median = df['Control Scores'].median()
control_mid_range = (df['Control Scores'].min() + df['Control Scores'].max()) / 2
control_range = df['Control Scores'].max() - df['Control Scores'].min()
control_iqr = np.percentile(df['Control Scores'], 75) - np.percentile(df['Control Scores'], 25)
control_population_variance = df['Control Scores'].var(ddof=0)
control_population_stddev = df['Control Scores'].std(ddof=0)

# Calculate statistics for 'Experimental Scores'
experimental_mean = df['Experimental Scores'].mean()
experimental_mode = df['Experimental Scores'].mode().iloc[0]
experimental_median = df['Experimental Scores'].median()
experimental_mid_range = (df['Experimental Scores'].min() + df['Experimental Scores'].max()) / 2
experimental_range = df['Experimental Scores'].max() - df['Experimental Scores'].min()
experimental_iqr = np.percentile(df['Experimental Scores'], 75) - np.percentile(df['Experimental Scores'], 25)
experimental_population_variance = df['Experimental Scores'].var(ddof=0)
experimental_population_stddev = df['Experimental Scores'].std(ddof=0)
```

DISCUSSION:

WHAT DO THESE SUMMARY STATISTICS TELL US?

```
Control Scores:
Mean: 78.08751663789478
Mode: 61.13590701261282 6
Median: 77.3861537433292 3
Mid-Range: 78.24753427263798
Range: 34.2232545200503
IQR: 15.577896589435682
Population Variance: 89.78049627551594
Population Standard Deviation: 9.47525705590703

Experimental Scores:
Mean: 85.31905514596525
Mode: 66.15394543335037
Median: 84.37171508463862
Mid-Range: 88.38986419909233
Range: 44.47183753148393
IQR: 17.49185513359525
Population Variance: 119.82727219584062
Population Standard Deviation: 10.946564401484176
```

LEARNING CHECK

Think about your answers to these questions:

- What are the different ways of identifying a ‘typical’ value?
- What does standard deviation tell us?
- Once outliers are identified, what might we do with them?

HOW DID YOU GET ON?

Learning objectives:

- Using an appropriate software tool, calculate:
  - Mean, Mode, Median, Mid-range
  - Population and Sample Standard Deviation & Variance
  - Inter-Quartile Range
- Discuss when the above measures are appropriate
- Apply methods for automating identification of outliers
- Discuss appropriate handling of outliers